

MILESTONE 2: CORE SEARCH ENGINE IMPLEMENTATION

A Technical Report on High-Performance Indexing & Intelligent Ranking

SEG301 — Search Engines & Information Retrieval

Project: Smart Price Comparison Bot

Team: phap-bot / SEG301_Project

1 Problem Statement

After Milestone 1, the team acquired **1,028,125 product records** from seven Vietnamese e-commerce platforms, producing a raw dataset of approximately **700MB**. This scale introduces two primary engineering challenges:

1. **Indexing at Scale:** A consumer laptop with limited RAM cannot load one million records simultaneously using traditional indexing approaches.
2. **Ranking with Accuracy:** Keyword matching alone cannot distinguish between valid products and noisy listings such as “Xác iPhone 15” or capacity mismatches like “iPhone 6 16GB” when searching for “iPhone 16”.

Milestone 2 addresses these challenges through a from-scratch implementation of **SPIMI indexing** and **BM25 ranking**.

2 Indexing Architecture: SPIMI Algorithm

2.1 Why SPIMI?

The *Single-Pass In-Memory Indexing (SPIMI)* algorithm enables indexing large corpora without exceeding RAM limits. Instead of maintaining a global term dictionary, SPIMI processes documents in independent blocks.

2.2 Implementation: Three-Phase Pipeline

Phase 1 — Block Processing

Documents are streamed from a JSONL file in blocks of **10,000 products**. Each block constructs an in-memory inverted index:

$$\text{Inverted Index} = \{\text{term} : \{\text{doc_id} : tf\}\}$$

Storing term frequency (tf) during indexing eliminates rescanning during search.

Phase 2 — K-Way Merge

Each block is sorted alphabetically and serialized into a binary `.pkl` file. A **priority queue (min-heap)** performs streaming merge across blocks, holding only one term per block in RAM.

Memory complexity:

$$O(K)$$

where K is the number of blocks.

Phase 3 — Metadata Persistence

Three persistent files are stored:

- `inverted_index.pkl`
- `doc_metadata.pkl`
- `doc_offsets.pkl`

2.3 Engineering Key: Byte Offsets for $O(1)$ Retrieval

During indexing, byte offsets are captured using `f.tell()`:

Naive Method	Proposed Method
Scan file line-by-line $O(N)$ retrieval	<code>f.seek(byte_offset)</code> $O(1)$ retrieval

2.4 Performance

Metric	Result
Documents Indexed	1,028,125
Unique Vocabulary	~314,042 terms
RAM Footprint	< 500MB
Indexing Time	< 10 minutes

3 Ranking System: BM25 Algorithm

3.1 Why BM25?

BM25 (Best Match 25) is a probabilistic ranking model widely used in production search engines such as Elasticsearch and Lucene. It improves upon TF-IDF through:

- Term-frequency saturation
- Document length normalization

3.2 BM25 Scoring Function

$$Score(D, Q) = \sum_{q \in Q} IDF(q) \cdot \frac{f(q, D)(k_1 + 1)}{f(q, D) + k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

3.2.1 Inverse Document Frequency

$$IDF(q) = \ln \left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1 \right)$$

Symbol	Meaning
N	Total documents (1,028,125)
$n(q)$	Documents containing term q

Rare terms receive higher weights, while common words approach zero contribution.

3.2.2 TF Saturation

$$TF_{sat} = \frac{f(q, D)(k_1 + 1)}{f(q, D) + K}$$

Using $k_1 = 2.0$:

$$\lim_{f(q, D) \rightarrow \infty} TF_{sat} = 3.0$$

This prevents keyword spam from dominating rankings.

3.2.3 Document Length Normalization

$$K = k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)$$

With $b = 0.8$ and $avgdl \approx 13.71$:

- Short titles $\rightarrow$ score boost
- Long titles $\rightarrow$ score penalty

4 Intelligent Reranking

After retrieving top-200 BM25 candidates, domain-specific heuristics rerank results.

4.1 Strategies

Strategy	Mechanism	Multiplier
Coordination Factor	Query coverage	$(match_ratio)^2$
Phrase Match Bonus	Exact phrase match	$\times 2.5$
Title Position Priority	Query at title start	$\times 1.5$
Noise Word Penalty	Accessory filtering	0.05^n
Unit vs Model Logic	Capacity mismatch	$\times 0.1$

4.2 Noise Word Penalty

Vietnamese marketplaces contain many accessory or scrap listings. A lexicon of 40+ noise tokens (e.g., “xac”, “op”, “cap”) penalizes titles beginning with such words:

$$Penalty_{front} = 0.05^{noise_count}$$

4.3 Model vs Capacity Disambiguation

If a numeric query token is followed by storage units:

$$\{gb, tb, mb, ram\}$$

the document receives a penalty:

$$\times 0.1$$

preventing confusion between model numbers and storage capacity.

5 System Performance Results

Metric	Result	Significance
Search Latency	0.05s–0.2s	Real-time response
P@10 Accuracy	High	Correct products top-ranked
Runtime Memory	<1GB	Fully RAM-resident index
Retrieval Cost	$O(1)$ I/O	Byte seek retrieval
Stability	No memory errors	1M+ documents indexed

6 Conclusion

Milestone 2 delivers a production-grade search engine implemented without third-party indexing or ranking libraries. SPIMI provides scalable indexing, while BM25 combined with heuristic reranking introduces domain-aware intelligence beyond generic search frameworks.

Prepared by Team phap-bot